SET-1:

1. a. Fibonacci Series

Situation

A biologist models a species where each month's population equals the sum of the previous two months. Print the population for the first n months.

Sample Input

```
8
```

Program

```
n = int(input())
a, b = 0, 1
series = []
for _ in range(n):
    series.append(a)
    a, b = b, a + b
print("Population growth:", *series)
```

Sample Output

```
Population growth: 0 1 1 2 3 5 8 13
```

1. b. Multiplication Table

Situation

A pricing engine needs a multiplier table up to 10 for a given rate. Read the rate and print its table.

Sample Input

```
7
```

Program

```
rate = int(input())
for i in range(1, 11):
    print(f"{rate} x {i} = {rate * i}")
```

Sample Output

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

1.c. Largest of Three Numbers

Situation

A broker compares the monthly rent of three properties. Display which property earns the most.

Sample Input

```
45000 52000 48500
```

Program

```
r1, r2, r3 = map(int, input().split())
if r1 >= r2 and r1 >= r3:
    print("Property 1 is highest:", r1)
elif r2 >= r1 and r2 >= r3:
    print("Property 2 is highest:", r2)
else:
    print("Property 3 is highest:", r3)
```

Sample Output

```
Property 2 is highest: 52000
```

2. Data Loading and Inspection

Situation

An analyst receives a raw transaction CSV. Load it, view the first rows, check the shape, data types and count missing values.

Sample Input

```
CSV text with columns CustomerID, Age, Amount, Category (defined in the program)
```

Program

```
import pandas as pd, io
csv = io.StringIO("""CustomerID,Age,Amount,Category
C1,25,1200,Grocery
C2,,850,Fashion
C3,41,,Grocery
C4,35,15000,Electronics
C5,29,640,Fashion
""")
df = pd.read_csv(csv)
print(df.head())
print("Shape:", df.shape)
print(df.dtypes)
print("Missing values:")
print(df.isnull().sum())
```

Sample Output

```
  CustomerID   Age   Amount     Category
0         C1  25.0   1200.0      Grocery
1         C2   NaN    850.0      Fashion
2         C3  41.0      NaN      Grocery
3         C4  35.0  15000.0  Electronics
4         C5  29.0    640.0      Fashion
Shape: (5, 4)
CustomerID        str
```

```
Age           float64
Amount        float64
Category          str
dtype: object
Missing values:
CustomerID    0
Age           1
Amount        1
Category      0
dtype: int64
```

Missing Value Imputation

Situation

Fill the missing Age with the column mean and the missing Amount with the column median so no analysis breaks.

Sample Input

```
Same CSV as part (a)
```

Program

```
import pandas as pd, io
df = pd.read_csv(io.StringIO("""CustomerID,Age,Amount,Category
C1,25,1200,Grocery
C2,,850,Fashion
C3,41,,Grocery
C4,35,15000,Electronics
C5,29,640,Fashion
"""))
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Amount"] = df["Amount"].fillna(df["Amount"].median())
print(df)
```

Sample Output

```
  CustomerID   Age   Amount     Category
0         C1  25.0   1200.0      Grocery
1         C2  32.5    850.0      Fashion
2         C3  41.0   1025.0      Grocery
3         C4  35.0  15000.0  Electronics
4         C5  29.0    640.0      Fashion
```

(c) Normalization

Situation

Scale the Amount column to the range 0 to 1 (min-max) and standardize Age (z-score) before modelling.

Sample Input

```
Amounts = 1200, 850, 1000, 15000, 640; Ages = 25, 32, 41, 35, 29
```

Program

```
import pandas as pd
df = pd.DataFrame({"Age": [25, 32, 41, 35, 29],
                   "Amount": [1200, 850, 1000, 15000, 640]})
```

```
df["Amount_minmax"] = (df["Amount"] - df["Amount"].min()) / (df["Amount"].max() -
df["Amount"].min())
df["Age_zscore"] = (df["Age"] - df["Age"].mean()) / df["Age"].std()
print(df.round(3))
```

Sample Output

```
   Age  Amount  Amount_minmax  Age_zscore
0   25    1200          0.039      -1.220
1   32     850          0.015      -0.066
2   41    1000          0.025       1.418
3   35   15000          1.000       0.429
4   29     640          0.000      -0.560
```

Outlier Removal

Situation

Remove transactions whose Amount lies outside 1.5 times the interquartile range (IQR) because they distort totals.

Sample Input

```
Amounts = 1200, 850, 1000, 15000, 640, 900
```

Program

```
import pandas as pd
df = pd.DataFrame({"Amount": [1200, 850, 1000, 15000, 640, 900]})
q1, q3 = df["Amount"].quantile(0.25), df["Amount"].quantile(0.75)
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
clean = df[(df["Amount"] >= low) & (df["Amount"] <= high)]
print("Bounds:", low, high)
print("Cleaned data:")
print(clean)
```

Sample Output

```
Bounds: 431.25 1581.25
Cleaned data:
   Amount
0    1200
1     850
2    1000
4     640
5     900
```

Data Visualization

Situation

Plot a bar chart of total spending per category and a line chart of monthly spending so executives can see trends and category shares.

Sample Input

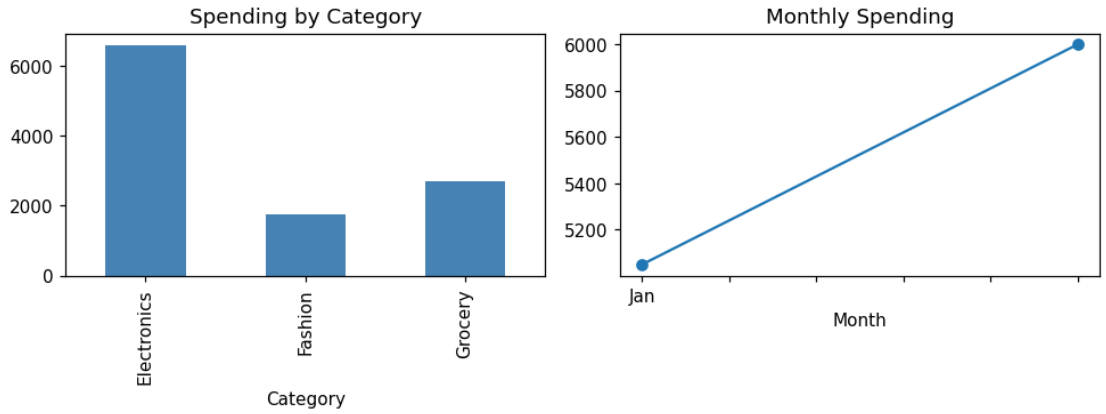
```
Category totals and monthly spends (defined in the program)
```

Program

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({"Category": ["Grocery", "Fashion", "Electronics", "Grocery", "Fashion",
"Electronics"],
                   "Month": ["Jan", "Jan", "Jan", "Feb", "Feb", "Feb"],
                   "Amount": [1200, 850, 3000, 1500, 900, 3600]})
totals = df.groupby("Category")["Amount"].sum()
monthly = df.groupby("Month", sort=False)["Amount"].sum()
print(totals)
fig, ax = plt.subplots(1, 2, figsize=(9, 3.5))
totals.plot(kind="bar", ax=ax[0], title="Spending by Category", color="steelblue")
monthly.plot(kind="line", marker="o", ax=ax[1], title="Monthly Spending")
plt.tight_layout()
plt.show()
```

Sample Output

```
Category
Electronics    6600
Fashion        1750
Grocery        2700
Name: Amount, dtype: int64
```



3.a. Block Matrix

Situation

An HVAC model combines four 2 x 2 zone matrices into one 4 x 4 composite matrix.

Sample Input

```
A = [[1, 2], [3, 4]], B = [[5, 6], [7, 8]], C = [[9, 10], [11, 12]], D = [[13, 14], [15, 16]]
```

Program

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = np.array([[9, 10], [11, 12]])
D = np.array([[13, 14], [15, 16]])
block = np.block([[A, B], [C, D]])
print(block)
```

Sample Output

```
[[ 1  2  5  6]
 [ 3  4  7  8]
 [ 9 10 13 14]
 [11 12 15 16]]
```

3.b. Reshape Matrix and Cube

Situation

A flat array of 12 sensor values must be reshaped into a 3 x 4 matrix and each element cubed.

Sample Input

```
arr = numbers 1 to 12  (defined in the program)
```

Program

```
import numpy as np
arr = np.arange(1, 13)
mat = arr.reshape(3, 4)
print("Reshaped matrix:")
print(mat)
print("Cube of elements:")
print(mat ** 3)
```

Sample Output

```
Reshaped matrix:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Cube of elements:
[[   1    8   27   64]
 [ 125  216  343  512]
 [ 729 1000 1331 1728]]
```

3.c. Row Sum and Column Sum

Situation

A finance team has a departmental expense matrix. Compute the total of every row and every column.

Sample Input

```
M = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]  (defined in the program)
```

Program

```
import numpy as np
M = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print("Row sums   :", M.sum(axis=1))
print("Column sums:", M.sum(axis=0))
```

Sample Output

```
Row sums   : [ 60 150 240]
Column sums: [120 150 180]
```

SET-2:

1. a. Standard Normal Distribution

Situation

A data scientist wants to standardize a dataset of heights (mean 170 cm, std 8 cm). Generate standard normal variables, convert the height data to z-scores and verify that the z-scores have mean about 0 and standard deviation about 1.

Sample Input

```
5000
```

Program

```
import numpy as np
np.random.seed(1)
n = int(input())
standard = np.random.randn(n)
heights = np.random.normal(170, 8, n)
z = (heights - heights.mean()) / heights.std()
print("Standard normal  -> mean:", round(standard.mean(), 4), "| std:", round(standard.std(),
4))
print("Heights          -> mean:", round(heights.mean(), 2), "| std:", round(heights.std(),
2))
print("Z-scores         -> mean:", round(z.mean(), 4) + 0, "| std:", round(z.std(), 4))
```

Sample Output

```
Standard normal  -> mean: 0.0243 | std: 1.0025
Heights          -> mean: 169.96 | std: 7.96
Z-scores         -> mean: 0.0 | std: 1.0
```

1. b. Exponential Distribution

Situation

A help desk sees customers arrive with an average gap of 2 minutes (exponential waiting time). Generate the waiting times and verify that the simulated mean is close to 2 minutes.

Sample Input

```
10000
```

Program

```
import numpy as np
np.random.seed(7)
n = int(input())
waits = np.random.exponential(scale=2, size=n)
print("Theoretical mean waiting time: 2.0 minutes")
print("Simulated mean waiting time   :", round(waits.mean(), 3), "minutes")
print("P(wait > 3 min) simulated:", round(np.mean(waits > 3), 4), "| exact:", round(np.exp(-3
/ 2), 4))
```

Sample Output

```
Theoretical mean waiting time: 2.0 minutes
Simulated mean waiting time   : 1.988 minutes
P(wait > 3 min) simulated: 0.224 | exact: 0.2231
```

1.c. Student's t Distribution

Situation

A quality engineer draws samples from a t distribution with 10 degrees of freedom. Compute the probability that a value exceeds 2 and compare it with the standard normal distribution to show the heavier tails of t.

Sample Input

```
100000
```

Program

```
import numpy as np
from scipy.stats import t, norm
np.random.seed(17)
n = int(input())
samples = np.random.standard_t(10, n)
print("t (df=10) simulated P(X > 2):", round(np.mean(samples > 2), 4))
print("t (df=10) exact     P(X > 2):", round(1 - t.cdf(2, 10), 4))
print("Normal exact        P(Z > 2):", round(1 - norm.cdf(2), 4))
print("Heavier tail in t distribution:", (1 - t.cdf(2, 10)) > (1 - norm.cdf(2)))
```

Sample Output

```
t (df=10) simulated P(X > 2): 0.0364
t (df=10) exact     P(X > 2): 0.0367
Normal exact        P(Z > 2): 0.0228
Heavier tail in t distribution: True
```

2. a. Prime Number Check Using a Function

Situation

A banking backend must verify that a security PIN is prime. Write is_prime(n) and test the submitted PIN.

Sample Input

```
97
```

Program

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

pin = int(input())
print(pin, "-> Valid (prime)" if is_prime(pin) else "-> Invalid (not prime)")
```

Sample Output

```
97 -> Valid (prime)
```

2. b. Perfect Number Check

Situation

A warehouse marks a tracking number as "perfect" if the sum of its proper divisors equals the number. Check the given number.

Sample Input

```
28
```

Program

```
n = int(input())
divisor_sum = sum(i for i in range(1, n) if n % i == 0)
print(n, "is a perfect number" if divisor_sum == n else "is not a perfect number")
```

Sample Output

```
28 is a perfect number
```

2. c. Second-Largest Element in a List

Situation

A contest portal wants the runner-up score from a list of candidate evaluations.

Sample Input

```
72 91 85 91 68 88
```

Program

```
scores = list(map(int, input().split()))
unique = sorted(set(scores), reverse=True)
if len(unique) >= 2:
    print("Runner-up score:", unique[1])
else:
    print("No second largest value")
```

Sample Output

```
Runner-up score: 88
```

3. a. Rank and Linear Dependence of Rows

Situation

Three sensor rows are recorded. Find the rank of the matrix and state whether the rows are linearly dependent.

Sample Input

```
M = [[1, 2, 3], [2, 4, 6], [1, 0, 1]]
```

Program

```
import numpy as np
M = np.array([[1, 2, 3], [2, 4, 6], [1, 0, 1]])
rank = np.linalg.matrix_rank(M)
print("Rank:", rank)
print("Rows are linearly", "independent" if rank == M.shape[0] else "dependent")
```

Sample Output

```
Rank: 2
Rows are linearly dependent
```

3. b. Non-Commutativity of Matrix Multiplication

Situation

Two operations, a rotation-like matrix A and a scaling matrix B, are applied in different orders. Show that A x B is not equal to B x A.

Sample Input

```
A = [[1, 2], [3, 4]], B = [[0, 1], [1, 0]]
```

Program

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [1, 0]])
print("A x B =")
print(A @ B)
print("B x A =")
print(B @ A)
print("Equal?", np.array_equal(A @ B, B @ A))
```

Sample Output

```
A x B =
[[2 1]
 [4 3]]
B x A =
[[3 4]
 [1 2]]
Equal? False
```

3. c. Block Diagonal Matrix and Its Determinant

Situation

A multi-zone structure is modelled by two independent 2 x 2 blocks placed on the diagonal of one big matrix. Build it and verify that its determinant equals the product of the block determinants.

Sample Input

```
B1 = [[2, 1], [1, 3]], B2 = [[4, 0], [0, 5]]
```

Program

```
import numpy as np
B1 = np.array([[2, 1], [1, 3]])
B2 = np.array([[4, 0], [0, 5]])
Z = np.zeros((2, 2))
big = np.block([[B1, Z], [Z, B2]])
print(big)
print("det(big)        :", round(np.linalg.det(big), 2))
print("det(B1) * det(B2):", round(np.linalg.det(B1) * np.linalg.det(B2), 2))
```

Sample Output

```
[[2. 1. 0. 0.]
 [1. 3. 0. 0.]
 [0. 0. 4. 0.]
 [0. 0. 0. 5.]]
det(big)        : 100.0
det(B1) * det(B2): 100.0
```

SET-3:

1. a. Filter Words That Contain a Digit

Situation

A message log must isolate any word that contains a digit.

Sample Input

```
order a1 shipped on day5 to room 12b safely
```

Program

```
words = input().split()
result = [w for w in words if any(c.isdigit() for c in w)]
print("Words with digits:", result)
```

Sample Output

```
Words with digits: ['a1', 'day5', '12b']
```

1. b. Uppercase Vowels and Lowercase Consonants

Situation

A stylist converts every vowel in a document to uppercase and every consonant to lowercase.

Sample Input

```
Programming Is FUN
```

Program

```
text = input()
out = ""
for ch in text:
    if ch.lower() in "aeiou":
        out += ch.upper()
    else:
        out += ch.lower()
print(out)
```

Sample Output

```
prOgrAmmIng Is fUn
```

1. c. First-and-Last-Letter Abbreviation

Situation

A tech team forms an acronym for each multi-word project title by taking the first and last letter of every word.

Sample Input

```
smart city project
```

Program

```
title = input().split()
abbr = "".join(w[0] + w[-1] for w in title).upper()
print("Abbreviation:", abbr)
```

Sample Output

```
Abbreviation: STCYPT
```

2. a. Matrix Slicing

Situation

From a 4 x 4 satellite image matrix, extract the central 2 x 2 region and the last column.

Sample Input

```
img = numbers 1 to 16 arranged as 4 x 4  (defined in the program)
```

Program

```
import numpy as np
img = np.arange(1, 17).reshape(4, 4)
print("Image:")
print(img)
print("Central 2x2 region:")
print(img[1:3, 1:3])
print("Last column:", img[:, -1])
```

Sample Output

```
Image:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
Central 2x2 region:
[[ 6  7]
 [10 11]]
Last column: [ 4  8 12 16]
```

2. b. Matrix Multiplication

Situation

Two design matrices A (2x3) and B (3x2) must be multiplied to find the combined loading forces.

Sample Input

```
A = [[1, 2, 3], [4, 5, 6]], B = [[7, 8], [9, 10], [11, 12]]
```

Program

```
import numpy as np
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[7, 8], [9, 10], [11, 12]])
print("A x B =")
print(np.dot(A, B))
```

Sample Output

```
A x B =
[[ 58  64]
 [139 154]]
```

2. c. Random Matrix Multiplication

Situation

A stochastic model multiplies two random 2 x 2 integer matrices (seed 7) to test noise behaviour.

Sample Input

```
seed = 7, values between 0 and 4
```

Program

```
import numpy as np
np.random.seed(7)
X = np.random.randint(0, 5, size=(2, 2))
Y = np.random.randint(0, 5, size=(2, 2))
print("X =", X.tolist())
print("Y =", Y.tolist())
print("X @ Y =")
print(X @ Y)
```

Sample Output

```
X = [[4, 1], [3, 3]]
Y = [[4, 1], [0, 1]]
X @ Y =
[[16  5]
 [12  6]]
```

3. a. Count Words Having Length Greater Than 6

Situation

An essay checker counts the long words (more than 6 characters) in a paragraph.

Sample Input

```
Programming requires patience and consistent practice every day
```

Program

```
words = input().split()
print("Long words:", sum(1 for w in words if len(w) > 6))
```

Sample Output

```
Long words: 5
```

3. b. Count Numbers with Repeated Digits

Situation

An audit script counts how many identification tokens contain at least one repeated digit.

Sample Input

```
112 345 998 1023 777 4821
```

Program

```
tokens = input().split()
count = sum(1 for t in tokens if len(set(t)) < len(t))
print("Tokens with repeated digits:", count)
```

Sample Output

```
Tokens with repeated digits: 3
```

3. c. Generate Pythagorean Triples

Situation

An architecture tool lists all right-angle triples (a, b, c) with a < b < c and c up to a given limit.

Sample Input

```
20
```

Program

```
limit = int(input())
for a in range(1, limit + 1):
    for b in range(a + 1, limit + 1):
        c = (a * a + b * b) ** 0.5
        if c == int(c) and c <= limit:
            print((a, b, int(c)))
```

Sample Output

```
(3, 4, 5)
(5, 12, 13)
(6, 8, 10)
(8, 15, 17)
(9, 12, 15)
(12, 16, 20)
```

SET-4:

1. a. Fibonacci Numbers Less Than n

Situation

A finance tool lists Fibonacci milestones that stay below a threshold n. Print all of them.

Sample Input

```
100
```

Program

```
n = int(input())
a, b = 0, 1
result = []
while a < n:
    result.append(a)
```

```
    a, b = b, a + b
print("Milestones below", n, ":", *result)
```

Sample Output

```
Milestones below 100 : 0 1 1 2 3 5 8 13 21 34 55 89
```

1. b. Armstrong Numbers in a Range

Situation

An encryption checker must list every Armstrong number between a lower and an upper limit.

Sample Input

```
100 500
```

Program

```
low, high = map(int, input().split())
found = []
for num in range(low, high + 1):
    s = str(num)
    if sum(int(d) ** len(s) for d in s) == num:
        found.append(num)
print("Armstrong numbers:", *found)
```

Sample Output

```
Armstrong numbers: 153 370 371 407
```

1. c. Merge Two Lists Without Duplicates

Situation

Two mailing lists must be combined into one list with no repeated address while keeping the first appearance order.

Sample Input

```
amy bob cat
bob dan amy eve
```

Program

```
list1 = input().split()
list2 = input().split()
merged = list1[:]
for item in list2:
    if item not in merged:
        merged.append(item)
print("Merged list:", merged)
```

Sample Output

```
Merged list: ['amy', 'bob', 'cat', 'dan', 'eve']
```

2. a. Random Matrix, Sums and Square

Situation

A simulator generates a random 3 x 3 matrix of integers (seed 42 for repeatable results), then prints its total sum, row sums and squared elements.

Sample Input

```
seed = 42, values between 1 and 9
```

Program

```
import numpy as np
np.random.seed(42)
R = np.random.randint(1, 10, size=(3, 3))
print("Matrix:")
print(R)
print("Total sum:", R.sum())
print("Row sums :", R.sum(axis=1))
print("Squares:")
print(R ** 2)
```

Sample Output

```
Matrix:
[[7 4 8]
 [5 7 3]
 [7 8 5]]
Total sum: 54
Row sums : [19 15 20]
Squares:
[[49 16 64]
 [25 49  9]
 [49 64 25]]
```

2. b. Block Matrix Using Identity and Zero Matrices

Situation

A routing security check needs a 4 x 4 block matrix with identity in the top-left, zeros in the top-right, zeros in the bottom-left and 2 times identity in the bottom-right.

Sample Input

```
(2 x 2 identity and zero matrices built in the program)
```

Program

```
import numpy as np
I = np.eye(2, dtype=int)
Z = np.zeros((2, 2), dtype=int)
block = np.block([[I, Z], [Z, 2 * I]])
print(block)
```

Sample Output

```
[[1 0 0 0]
 [0 1 0 0]
 [0 0 2 0]
 [0 0 0 2]]
```

2. c. Square and Cube of Matrix Elements

Situation

A materials lab has a stress grid. Compute the element-wise square and cube of the matrix.

Sample Input

```
A = [[1, 2], [3, 4]]  (defined in the program)
```

Program

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
print("Square:")
print(A ** 2)
print("Cube:")
print(A ** 3)
```

Sample Output

```
Square:
[[ 1  4]
 [ 9 16]]
Cube:
[[ 1  8]
 [27 64]]
```

3. a. Poisson Distribution

Situation

A store receives on average 4 customers per hour (Poisson, lambda = 4). Estimate the probability that more than 6 customers arrive in an hour by simulation and compare it with the exact value.

Sample Input

```
100000
```

Program

```
import numpy as np
from scipy.stats import poisson
np.random.seed(3)
n = int(input())
arrivals = np.random.poisson(lam=4, size=n)
print("Average arrivals per hour:", round(arrivals.mean(), 3))
print("P(X > 6) simulated:", round(np.mean(arrivals > 6), 4))
print("P(X > 6) exact    :", round(1 - poisson.cdf(6, 4), 4))
```

Sample Output

```
Average arrivals per hour: 4.0
P(X > 6) simulated: 0.1104
P(X > 6) exact    : 0.1107
```

3. b. Binomial Distribution

Situation

A game club tosses a fair coin 10 times in each experiment and repeats it 1000 times. Simulate this and find the most frequent number of heads.

Sample Input

```
1000
```

Program

```
import numpy as np
np.random.seed(0)
experiments = int(input())
heads = np.random.binomial(n=10, p=0.5, size=experiments)
counts = np.bincount(heads, minlength=11)
for h, c in enumerate(counts):
    print(f"{h:2d} heads : {c}")
print("Most frequent number of heads:", counts.argmax())
```

Sample Output

```
 0 heads : 1
 1 heads : 6
 2 heads : 40
 3 heads : 128
 4 heads : 217
 5 heads : 245
 6 heads : 182
 7 heads : 121
 8 heads : 50
 9 heads : 8
10 heads : 2
Most frequent number of heads: 5
```

3. c. Beta Distribution

Situation

A marketing team is uncertain about a conversion rate and models it with a Beta distribution with alpha = 2 and beta = 5. Generate samples and compute the expected probability.

Sample Input

```
10000
```

Program

```
import numpy as np
np.random.seed(19)
n = int(input())
a, b = 2, 5
p = np.random.beta(a, b, n)
print("Theoretical expected probability: a/(a+b) =", round(a / (a + b), 4))
print("Simulated expected probability  :", round(p.mean(), 4))
print("Range of generated values: min =", round(p.min(), 3), "max =", round(p.max(), 3))
```

Sample Output

```
Theoretical expected probability: a/(a+b) = 0.2857
Simulated expected probability  : 0.284
Range of generated values: min = 0.005 max = 0.868
```